Ex 10: Applying Performance Optimization Techniques (Indexing, Subforms, Query Tuning) to a Zoho Creator Application

Topic: Patient Appointment Booking Form

V Vijay Prakash
241401128

1. Aim

To apply and evaluate performance optimization techniques — well-chosen search fields, subform-based data design and Deluge query tuning — so that data retrieval and form submission stay fast as the Patient Appointment Booking application grows.

2. Procedure

Step 1: Choose efficient search and filter fields — The fields most often used to look up bookings were identified: Doctor, Appointment_Date, Time_Slot, Phone and Email. These were used as the criteria for report filters, quick filters and fetch statements, so that lookups narrow the data on specific values instead of scanning every record.

Step 2: Review the use of subforms — Where a booking needs several related line entries, the design options were compared: keeping them in a separate child form linked by a lookup, versus embedding them as a subform inside the parent form so the parent and its entries are submitted together in a single operation.

Step 3: Tune the Deluge queries — The booking validation script was reviewed for avoidable cost:

241401128

- **Criteria in the query:** the clash check places doctor, date and time slot inside the fetch criteria, so only matching records are returned.
- **No fetching inside loops:** repeated fetch calls inside a for each loop are replaced by one fetch with a combined criteria.
- **Count instead of fetch-and-loop:** where only the number of matches is needed, count() is used instead of reading every record.
- **Limit what is retrieved:** reports show only the columns needed, and fetches use a range when only a few records are required.

3. Output

3.1 Deluge Query Patterns: Before and After Tuning

The same duplicate-booking check written in a slower and a tuned form:

Unoptimised pattern — fetches every record and tests each one in a loop:

```
all = Patient_Appointment_Booking[ID != 0];
for each r in all
{
    if(r.doctor == input.doctor && r.Appointment_Date1 == input.Appointment_Date1
       && r.time_slot == input.time_slot)
    {
        alert "This slot is already booked. Please choose another.";
        cancel submit;
    }
}
```

Tuned pattern — criteria are inside the query and only a count is read:

```
clash = Patient_Appointment_Booking[doctor == input.doctor
        && Appointment_Date1 == input.Appointment_Date1
        && time_slot == input.time_slot];
if(clash.count() > 0)
{
    alert "This slot is already booked. Please choose another.";
    cancel submit;
}
```

3.2 Summary of Optimization Techniques

Technique	Applied To	Expected Benefit	Observed Result
Criteria on specific fields	Reports, filters, fetch statements (Doctor, Appointment_Date, Time_Slot)	Narrows the records examined for each lookup	
Subform for related entries	Parent booking with multiple line entries	One combined submission instead of separate parent and child saves	
Criteria inside the fetch	Duplicate-booking check in Validate appointment	Only matching records are returned, no per-record looping	
count() instead of loops	Clash check	Avoids reading full records when only a number is needed	

Table 1: Optimization Techniques Applied to the Appointment Booking Application

2211401128

4. Result / Observations

Performance optimization techniques were applied to the Patient Appointment Booking application in Zoho Creator:

1. Using specific, frequently searched fields as filter and fetch criteria limits how much data each lookup has to examine.
2. Embedding related entries in a subform lets a parent record and its entries be submitted together instead of as separate saves.
3. Moving criteria into the fetch statement and using count() removed looping over all records in the duplicate-booking check.